

{creative} [coding]

Dokumentation Woche 1

{Montag} 27.04.2026

Vormittag

- Allgemeine Einführung in die Colabor-Module
- Vorstellung eines Klangkünstlers

Nachmittag

- Vorstellung des Moduls
 - Kurze persönliche Vorstellung
 - Austausch über Programmierkenntnisse und Erwartungen
 - Einteilung in Gruppen
-

{Dienstag} 28.04.2026

Vormittag

- Einführung in P5.live

Nachmittag

- Beginnend mit der Nachbildung von Vera Molnars generativer Kunst aus Quadraten
- Einführung in die „for“-Schleife
- Einführung in die Zufallsfunktion

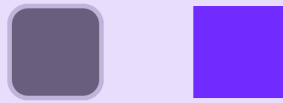
☆ BASIC PAINT

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  background(171, 255, 178)  
  //frameRate(200)  
}  
  
function draw() {  
  //paint program very basic  
  background(171, 255, 178)  
  ellipse(mouseX, mouseY, 50)  
}
```



☆ BASIC SQUARE

```
function setup() {  
  createCanvas(windowWidth, windowHeight)  
  background(232, 222, 252)  
}  
  
function draw() {  
  // left square with border-radius  
  fill(105, 95, 125)  
  stroke(193, 180, 219)  
  strokeWeight(5)  
  square(300, 200, 100, 20)  
  
  // right square  
  fill(114, 43, 255)  
  noStroke()  
  square(500, 200, 100)  
}
```



☆ VERSUCHE NACHBILDUNG VON VERA MOLNARS

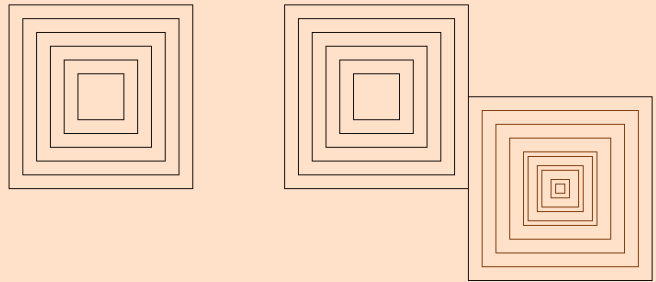
```
function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255, 224, 201)
  rectMode(CENTER)
}

let dim = 200
let reduction = 30
let posX = 800
let posY = 200

function draw() {
  noFill()
  strokeWeight(1)
  stroke(0)
  // basic variant square in square
  square(posX, posY, dim)
  square(posX, posY, dim - (reduction * 1))
  square(posX, posY, dim - (reduction * 2))
  square(posX, posY, dim - (reduction * 3))
  square(posX, posY, dim - (reduction * 4))
  square(posX, posY, dim - (reduction * 5))

  //for loop - multiple square
  for(let i = 0; i < 6; i++) {
    square(posX+300, posY, dim - (reduction * i))
  }
  // for loop - brown-squares
  for(let i = 0; i < 10; i++) {
    square(posX+500, posY+100, dim - (reduction * i))
    stroke(117, 50, 0)
  }
}
```

```
}
```



☆ TEST ANIMATION

```
let dimX = 450
let dimY = dimX
let num = 17
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(141, 186, 165)
  // The rectangle is drawn from the center
  rectMode(CENTER)
  angleMode(DEGREES)
  // Position in the center of the screen
  posX = width / 2
  posY = height / 2
}

function draw() {
  // Redraw the background, important for animation
  background(141, 186, 165)
  noFill()
  strokeWeight(1)
  stroke(0)
  // Animation: Width changes due to
  // frameCount increases → pulsating movement
  dimX = (sin(frameCount * 4) * 300)
  // number of rectangles
  num = 20
```

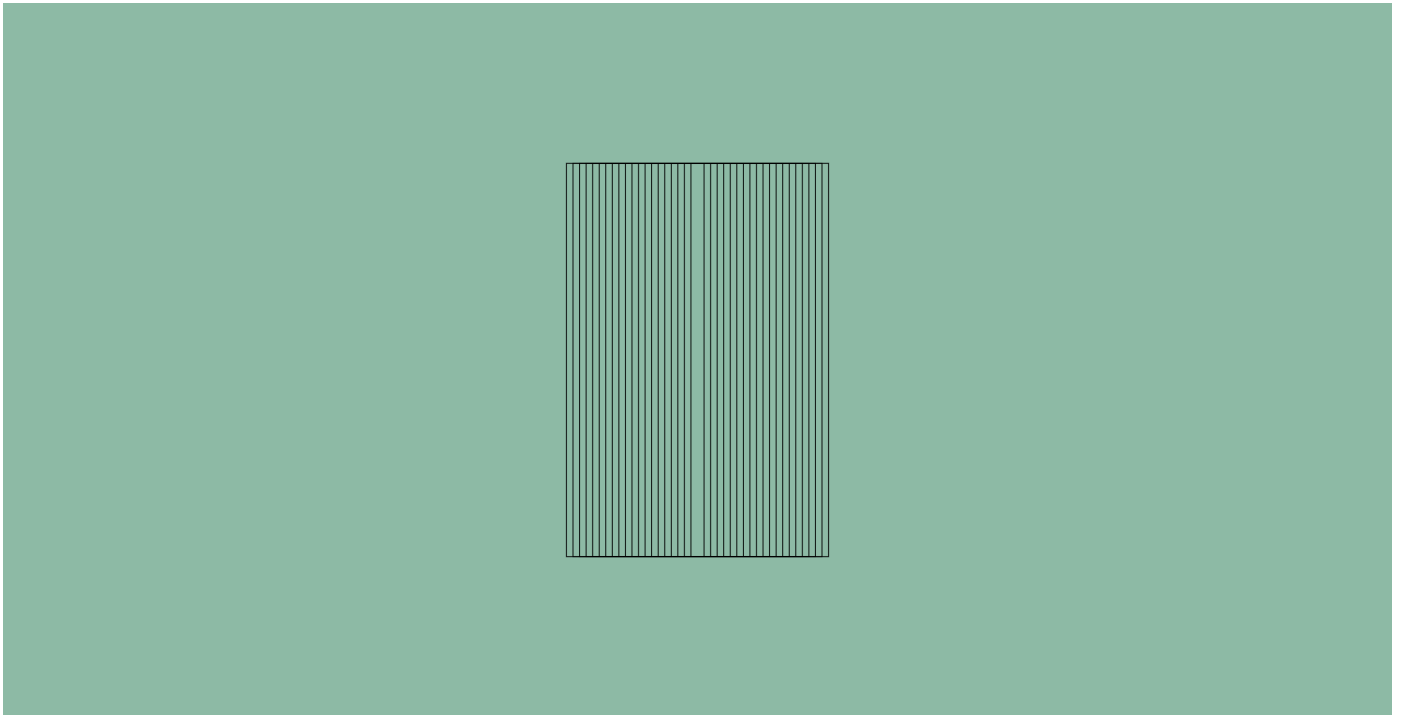


```

// Recalculating distance between sizes
reduction = dimX / num

// for-loop - draws multiple squares
for(let i = 0; i < num; i++) {
  rect(posX, posY,
    (dimX) - (reduction * i),
    (dimY) - (dimY/num * 1))
}
//(text(frameCount, 400, 400)
}

```



☆ VERSUCHE NACHBILDUNG VON VERA MOLNARS (for-Loops)

```

let dimX = 450
let dimY = dimX
let num = 17
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(141, 186, 165)
  // The rectangle is drawn from the center
  rectMode(CENTER)
  angleMode(DEGREES)
  // Position in the center of the screen
  posX = width / 2
  posY = height / 2
}

function draw() {

```

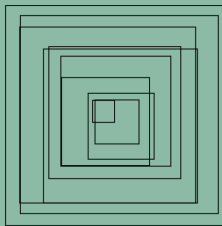
```

// HG neu fÄzllen fÄžr Animation
background(141, 186, 165)
noFill()
strokeWeight(1)
stroke(0)

// animation - Wellenbewegung
// calculate the dimension of the square
// based on a sin function
dimX = 200 + (sin(frameCount * 4) * 50)
// Anz Quadrate
num = 10
// Berechnung Verkleinerung Quadrate
reduction = dimX / num

// for-loop -> zeichnet mehrere Quadrate
for(let i = 0; i < num; i++) {
  // ZufÄillige Verschiebung 0-20px
  let offsetX = random(20)
  let offsetY = random(20)
  // quadrate werden gezeichnet, immer kleiner
  square(
    posX + offsetX,
    posY + offsetY,
    (dimX) - (reduction * i),
  )
}
//(text(frameCount, 400, 400)
//noLoop()
}

```



{Mittwoch} 29.04.2026

Vormittag

- Weiterführung der Nachbildung von Vera Molnars Kunstwerk
- Einführung in die „if“-Funktion
- Wie man eigene Funktionen erstellt

Nachmittag

- Einführung in den 3D-Raum in P5.live
- Nachbildung eines DVD-Bildschirmschoners in 3D

★ NACHBILDUNG VON VERA MOLNARS

```
let dimX = 453
let dimY = dimX
let num = 17
let reduction = dimX / num
let posX = 0
let posY = 0

function setup() {
  createCanvas(windowWidth, windowHeight)
  background(255)
  // Rechtecke von Mitte aus zeichnen
  rectMode(CENTER)
  angleMode(DEGREES)
  posX = width / 2
  posY = height / 2
}

function draw() {
  // HG-Farbe neu gesetzt -> Animation
  background(115, 0, 54)
  noFill()
  strokeWeight(3)
  stroke(0)
  // nested loops
  // Anz Spalten
  let numX = 10
  // 1 loop - Spalten
  for(let i = 0; i < numX; i++){
    // Breite eines Grid-Feldes
    let dimension = width / numX
    // X-Pos pro Spalte
    let posX = dimension / 2 + (i * dimension)
    // 2 loop - Reihen

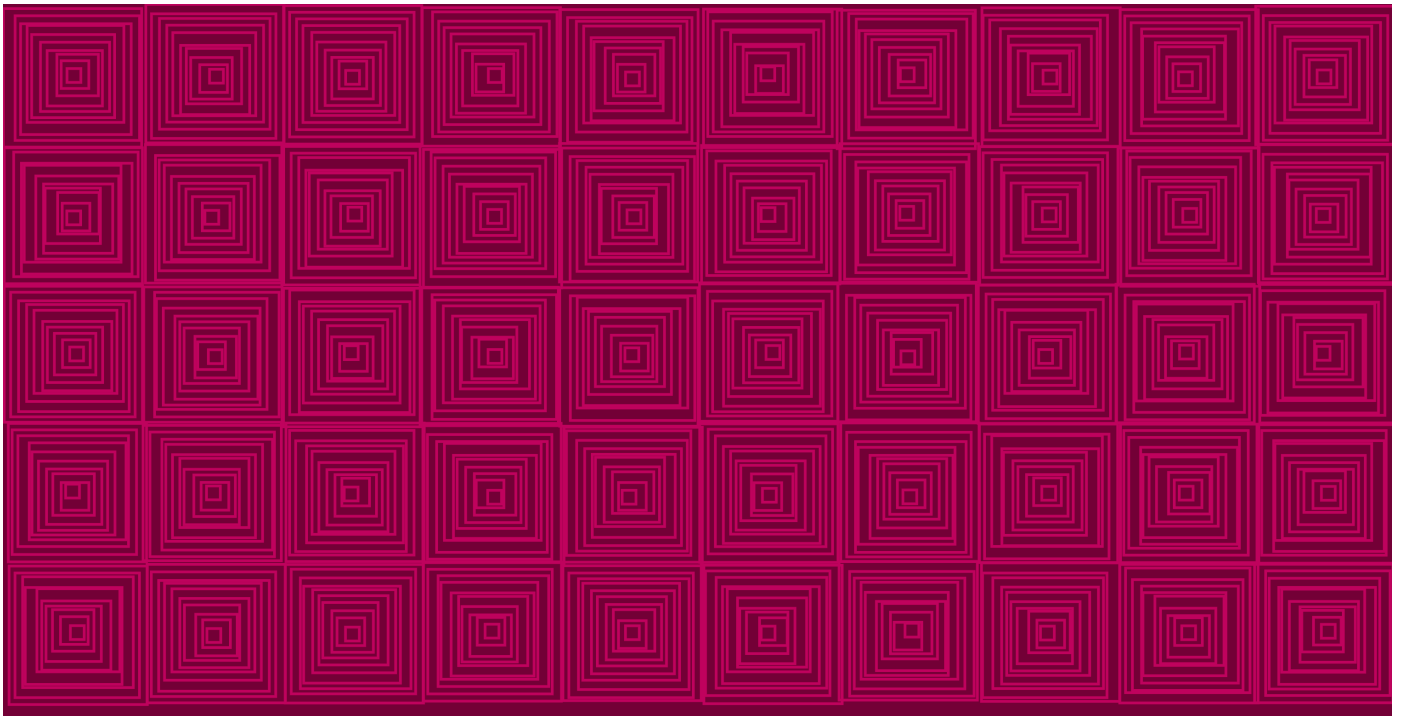
    for(let j = 0; j < 5; j++){
      // Y-Pos pro Reihe
      let posY = dimension / 2 + (j * dimension)
      // Funktion fÄŕ jede Grid-Zelle
      tmc(posX, posY, dimension, 1, 10)
```

```

    }
  }
}

// Funktion zum Zeichnen der Quadrate
function tmcs(x, y, dim, speed, num){
  // Animation Pulsieren
  let dimension = dim + sin(frameCount * speed) * 10
  // Berechnung Verkleinerung der Quadrate
  let reduction = dimension / num
  // loop - mehrere quadrate ineinanderf
  for(let i = 0; i < num; i++) {
    // zufällige verschiebung für zitter-effekt
    let offsetX = random(7)
    let offsetY = random(7)
    stroke(191, 2, 93)
    strokeWeight(3)
    // Quadrate zeichnen
    square(
      x+offsetX,
      y+offsetY,
      (dimension) - (reduction * i)
    )
  }
}
//noLoop()
}

```



★ 3D WÜRFEL AUF ACHSE

```

let posX = 0
let posY = 0
let boxDim = 100

function setup() {

```

```

// WEBGL = 3D
createCanvas(windowWidth, windowHeight, WEBGL)

}

function draw() {
  background(0)
  // Maussteuerung der Kamera (Sicht von Kamera)
  orbitControl()
  // Füllung Würfel
  fill(255)

  // Bewegung x-Achse
  posX++
  // Geschwindigkeit
  posX+=10
  // Bewegung y-Achse
  posY = posY - 1

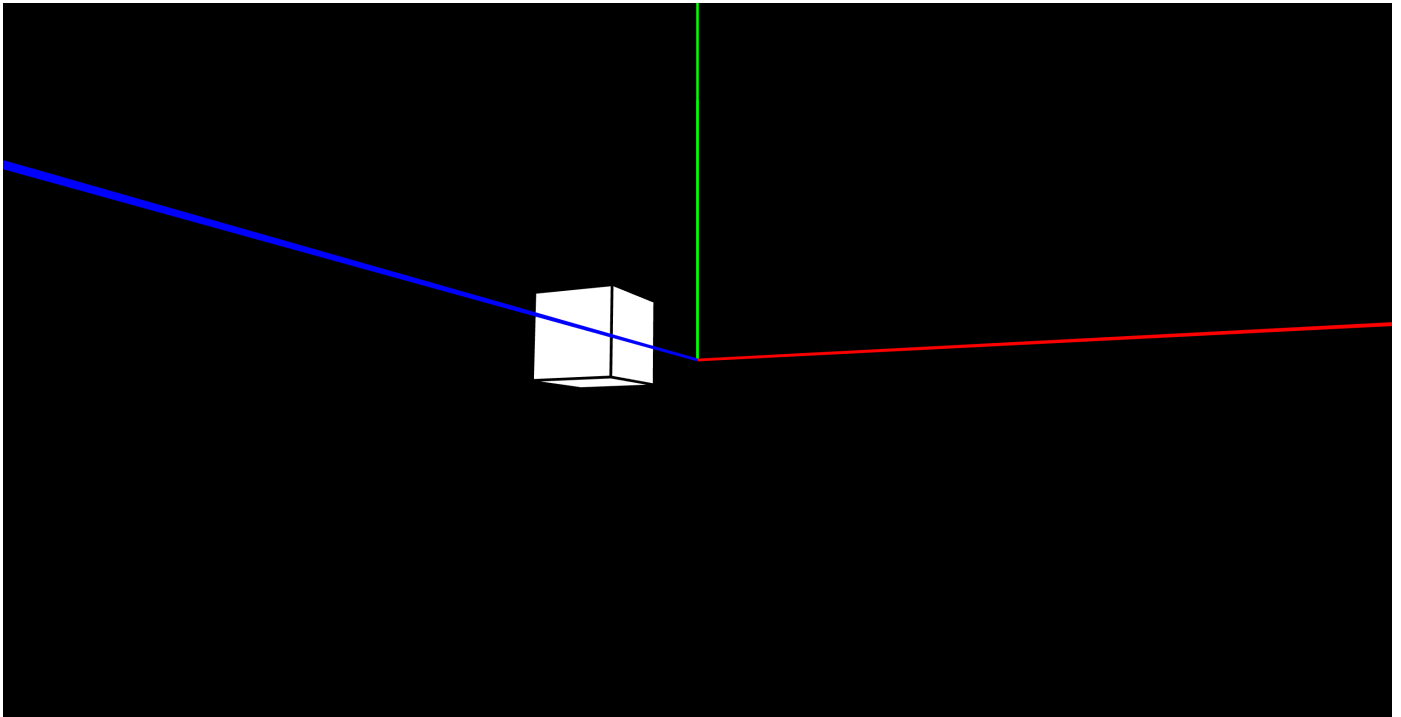
  // Wenn Würfel rechts aus Bild geht,
  // wieder links starten
  if(posX > width/2 + boxDim){
    posX = -width/2
  }

  // Wenn Würfel zu weit nach oben geht,
  // zurücksetzen
  if(posY < -(height / 2)){
    posY = 0
  }

  // Transformation starten!
  push()
  // Würfel an neue Pos verschieben
  translate(posX, posY, 0)
  // Würfel zeichnen
  box(boxDim)
  // Transformation beenden!
  pop()

  // 3D-Achsen zeichnen (rot, blau, grün)
  strokeWeight(3)
  push()
  stroke(255, 0, 0)
  line(0,0,0, width, 0, 0)
  stroke(0, 255, 0)
  line(0,0,0, 0, -height, 0)
  stroke(0, 0, 255)
  line(0,0,0, 0, 0, 1000)
  pop()
}

```



★ 2D ANIMATION INSPIRED BY DVD-SCREENSAVER

```
let x, y;
let xspeed, yspeed;
let r, g, b;

// Grösse Rechteck
const rectW = 80;
const rectH = 60;

function setup() {
  // WEBGL: Ursprung in Mitte
  createCanvas(windowWidth, windowHeight, WEBGL);
  // Zufällige Startposition auf Bildschirm
  x = random(width - rectW);
  y = random(height - rectH);
  // Geschwindigkeit
  xspeed = 6;
  yspeed = 6;
  // Anfangsfarbe setzten
  pickcolor();
}

// Funktion zufällige Farbe
function pickcolor() {
  r = random(100, 255);
  g = random(100, 255);
  b = random(100, 255);
}

// Verschiebt Koordinatensystem
// von Mitte nach oben links
function draw() {
  translate(-width / 2, -height / 2);
```

```

background(0);

noStroke();
fill(r, g, b);
rect(x, y, rectW, rectH);

// Position pro Frame verändern
x += xspeed;
y += yspeed;

// ----- Aufprall X-Achse -----
// rechter Rand
if (x + rectW >= width) {
    xspeed *= -1; // Richtung umkehren
    x = width - rectW; // Korrigieren, nicht aus Bild
    pickcolor(); // neue Farbe
}
// linker Rand
else if (x <= 0) {
    xspeed *= -1;
    x = 0;
    pickcolor();
}

// ----- Aufprall Y-Achse -----
// Unterer Rand
if (y + rectH >= height) {
    yspeed *= -1;
    y = height - rectH;
    pickcolor();
}
// Oberer Rand
else if (y <= 0) {
    yspeed *= -1;
    y = 0;
    pickcolor();
}
}

```



★ 3D ANIMATION INSPIRED BY DVD-SCREENSAVER

```
// Position 3D-Raum
let x;
let y;
let z;
// Geschwindigkeit pro Richtung
let xspeed;
let yspeed;
let zspeed;
// Raumentiefe
let depth = 2000;
// Farben
let r, g, b;

// WEBGL = 3D & Ursprung in Mitte
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL);
  // Zufällige Startposition auf Bildschirm
  x = random(width);
  y = random(height);
  z = random(-depth, 0);
  // Geschwindigkeit
  xspeed = 4;
  yspeed = 4;
  zspeed = 4;
  // Anfangsfarbe setzten
  pickcolor();
}

// Funktion zufällige Farbe
function pickcolor() {
  r = random(255);
  g = random(255);
```



```

    b = random(255);
}

// Verschiebt Koordinatensystem
// von Mitte nach oben links
function draw() {
    translate(-width/2, -height/2)
    background(0);
    fill(255);
    fill(r, g, b);

    // Transformation starten!
    push()
    // Würfel an Pos verschieben
    translate (x, y, z)
    // Würfel zeichnen
    box (100)
    // Transformation beenden!
    pop()

    // ----- Bewegung -----
    x = x + xspeed; // rechts / links
    y = y + yspeed; // unten / oben
    z = z + zspeed; // tiefe

    // ----- Aufprall Z-Achse -----
    // vor
    if (z >= 100) {
        zspeed = -zspeed; // Richtung umkehren
        z = 100; // Pos korrigieren, nicht aus Bild
        pickcolor(); // neue Farbe
    }
    // zurück
    else if (z <= -depth) {
        zspeed = -zspeed;
        z = -depth;
        pickcolor();
    }

    // ----- Aufprall X-Achse -----
    if (x >= width) {
        xspeed = -xspeed;
        x = width;
        pickcolor();
    }
    else if (x <= 0) {
        xspeed = -xspeed;
        x = 0;
        pickcolor();
    }

    // ----- Aufprall Y-Achse -----
    if (y >= height) {
        yspeed = -yspeed;
        y = height;
        pickcolor();
    }
    else if (y <= 0) {

```

```

    yspeed = -yspeed;
    y = 0;
    pickcolor();
  }
}

```



★ 3D ANIMATION INSPIRED BY DVD-SCREENSAVER MIT SICHTBAREM RAUM

```

// Position 3D-Raum
let x;
let y;
let z;

// Geschwindigkeit pro Richtung
let xspeed;
let yspeed;
let zspeed;
// Raumentiefe
let depth = 2000;
// Farbe
let r, g, b;

// WEBGL = 3D! & Ursprung in Mitte
function setup() {
  createCanvas(windowWidth, windowHeight, WEBGL);
  // Zufällige Startposition auf Bildschirm
  x = random(width);
  y = random(height);
  z = random(-depth, 100);
  // Geschwindigkeit
  xspeed = 4;
  yspeed = 4;
  zspeed = 4;
  // Anfangsfarbe setzen

```

```

    pickcolor();
}

// Funktion zufällige Farbe
function pickcolor() {
    r = random(255);
    g = random(255);
    b = random(255);
}

function draw() {
    background(255, 255, 255);
    // Kamera bewegen
    orbitControl();
    // Licht für 3D-Optik
    ambientLight(100);
    pointLight(255, 255, 255, 0, 0, 500);

    // ----- RAUM-WÜRFEL -----
    push();
    noFill();           // nur Kanten sichtbar
    stroke(100);        // graue Linien
    strokeWeight(1);
    box(width, height, depth);
    pop();

    // ----- BEWEGTE BOX -----
    noStroke();
    fill(r, g, b);
    // Transformation starten!
    push();
    // Würfel an Pos verschieben
    translate(x - width / 2, y - height / 2, z);
    // Würfel zeichnen
    box(100);
    // Transformation beenden!
    pop();

    // ----- BEWEGUNG -----
    x += xspeed;
    y += yspeed;
    z += zspeed;

    // ----- Aufprall Z-Achse -----
    if (z >= depth / 2) {
        zspeed *= -1;
        z = depth / 2;
        pickcolor();
    }
    else if (z <= -depth / 2) {
        zspeed *= -1;
        z = -depth / 2;
        pickcolor();
    }

    // ----- Aufprall X-Achse -----
    if (x >= width) {
        xspeed *= -1;
    }

```

```
x = width;
pickcolor();
}
else if (x <= 0) {
  xspeed *= -1;
  x = 0;
  pickcolor();
}

// ----- Aufprall >-Achse -----
if (y >= height) {
  yspeed *= -1;
  y = height;
  pickcolor();
}
else if (y <= 0) {
  yspeed *= -1;
  y = 0;
  pickcolor();
}
}
```



{Donnerstag} 30.04.2026

Vormittag

- Kurze Einführung in Sound-Design-Programme: Strudel, VCV Rack
- Einführung in VCV Rack 2

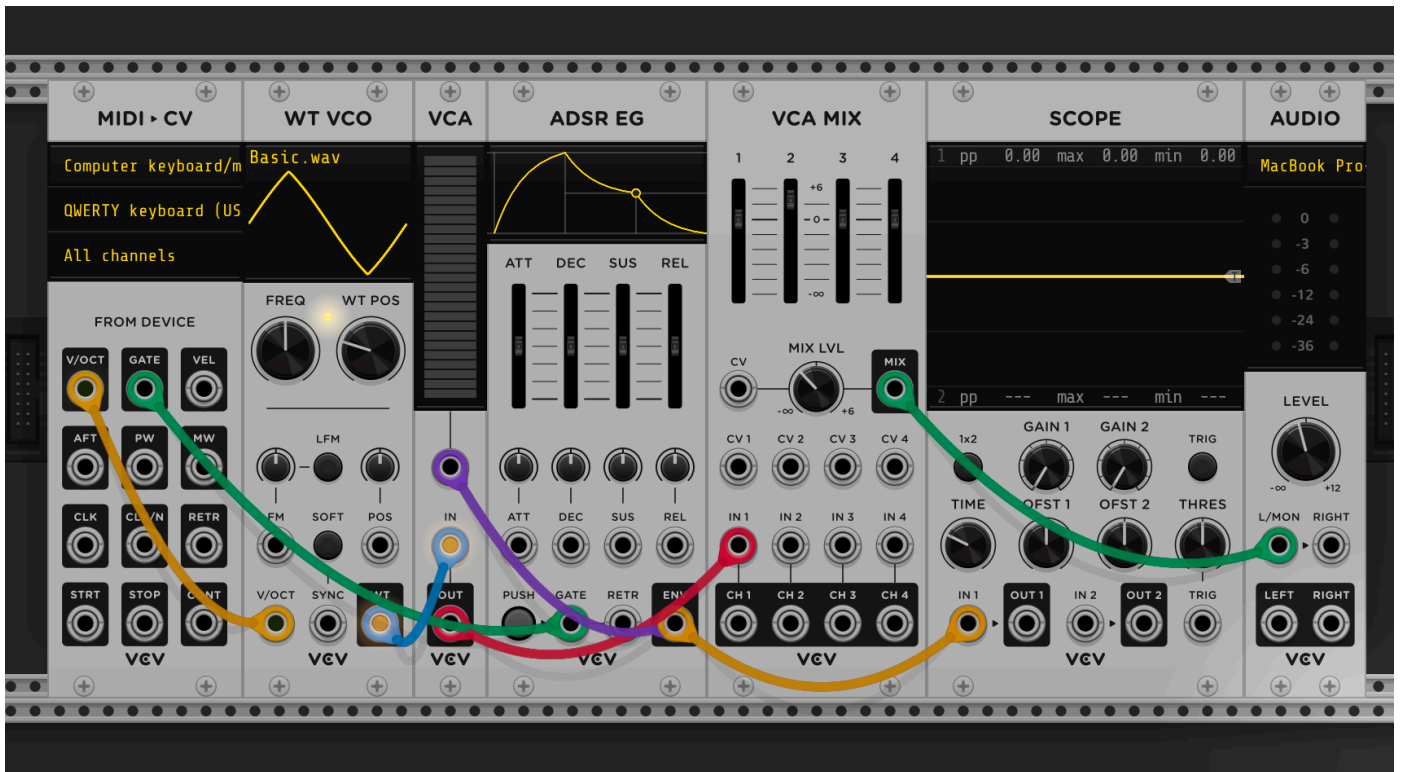
Nachmittag

- abwesend wegen der Einführung in den 3D-Druck-Workshop

☆ BASIC SIREN-LIKE SOUND



☆ KEYBOARD SOUND SYNTHESIZER



{Freitag} 01.05.2026

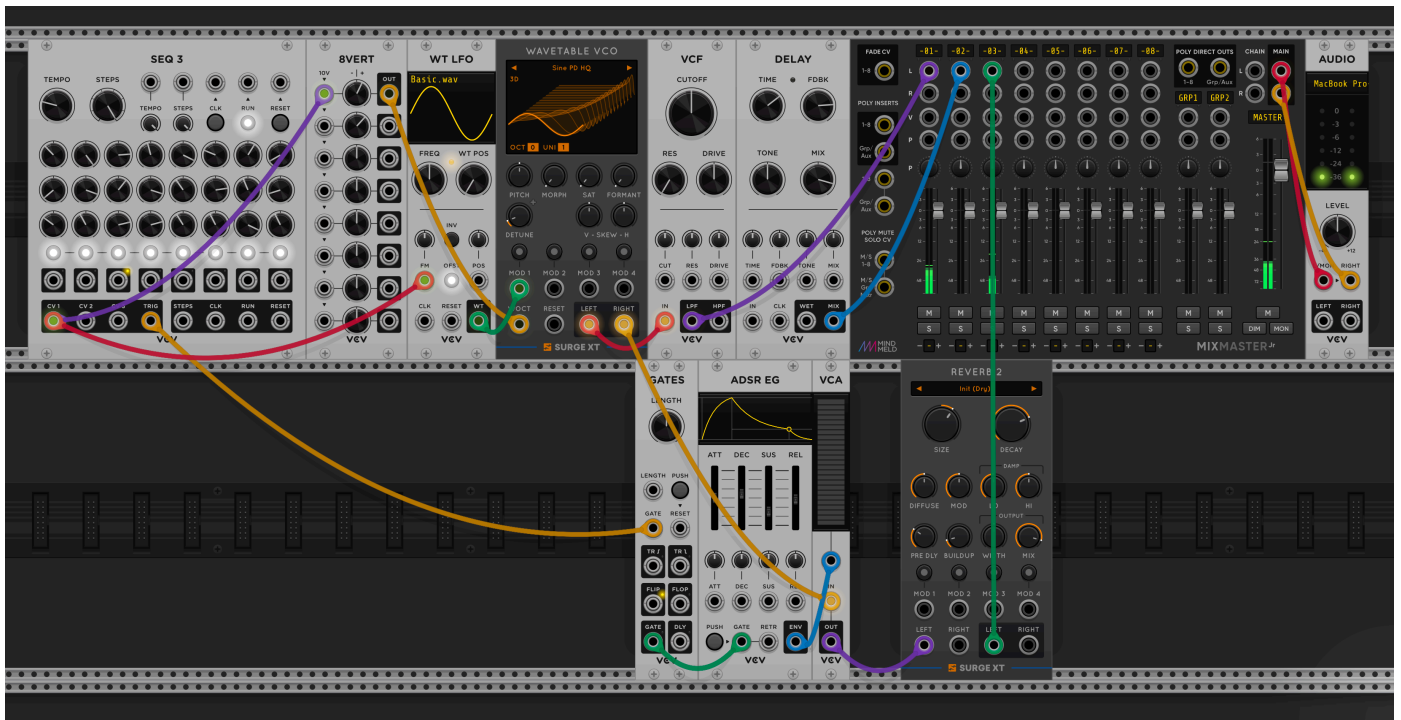
Vormittag

- Inspiration für generative Kunst / Live-Coding sammeln
- Telefonat mit Yann:
→Allgemeine Klärung des Endprodukts / Ziels und der Zwischenpräsentation
- Nachholen, was ich am Donnerstag verpasst habe
- Experimente mit P5.LIVE

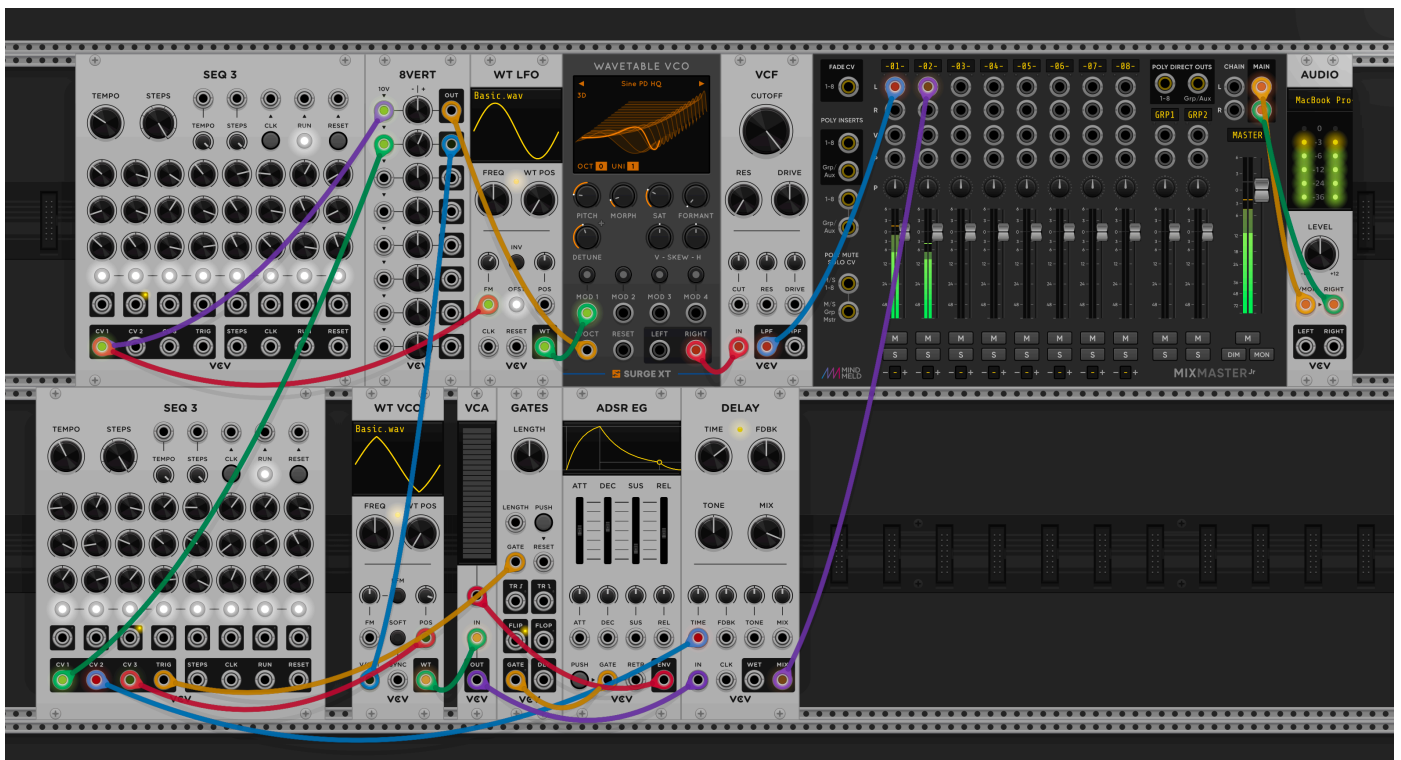
★ Experiment 3



★ Experiment 4



★ Experiment 5



★ P5.LIVE – Experiment 1

```
function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // halb transparenter HG -> "Spuren"
```

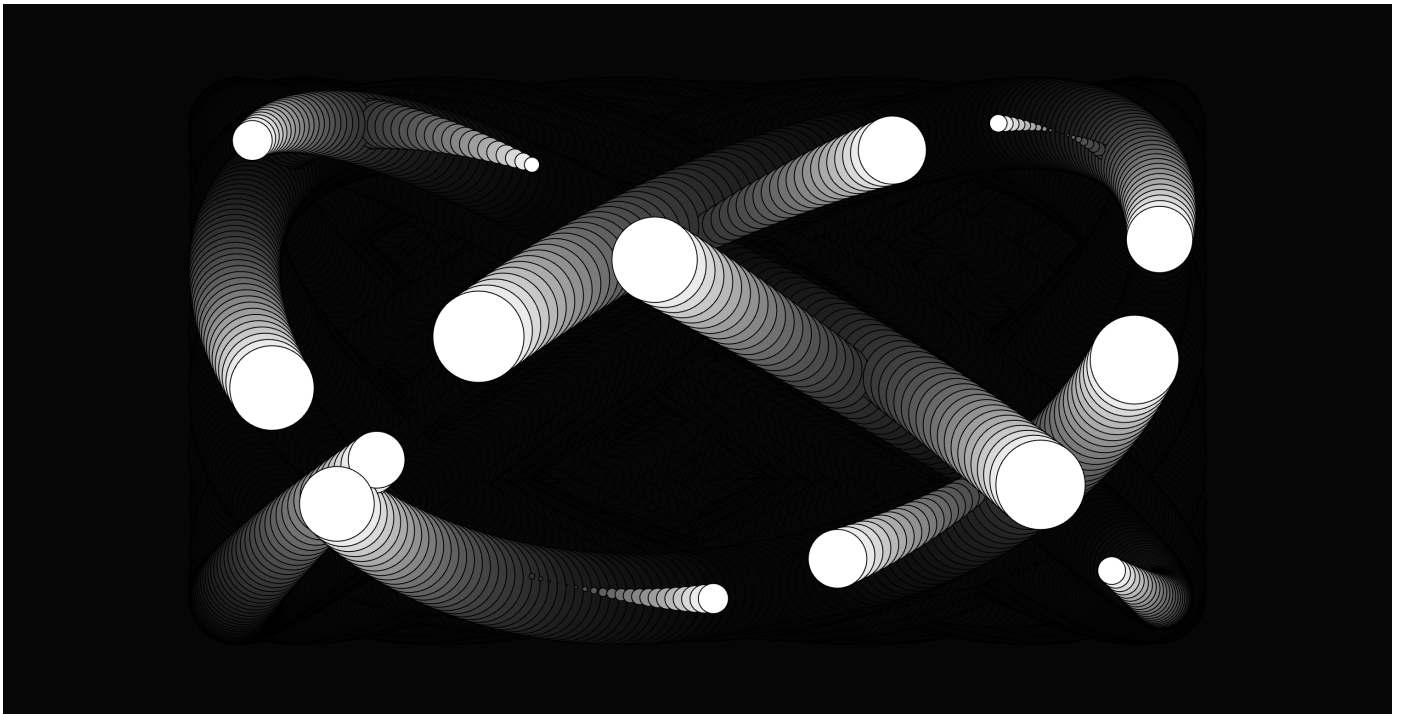
```

background(0, 20)

// Anz Kreise
let lc = 15;

// for-loop > zeichnet mehrere Kreise
for(let i = 0; i < lc; i++) {
  // X-Pos mit Sinus > frameCount = Animation über Zeit
  let x = sin(i * .4 + frameCount * .02) * width / 3;
  // Y-Pos mit Cosinus > frameCount = Animation über Zeit
  let y = cos(i * 2.6 + frameCount * .025) * height / 3;
  // Grösse der Animation
  let s = sin(i * .5 + frameCount * .02) * 100;
  // Zeichnet Ellipse
  ellipse(width / 2 + x, height / 2 + y, s)
}
}

```



★ P5.LIVE – Experiment 2

```

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // leicht transparenter HG > Spuren
  background(0, 15)
  stroke(255)
  noFill()

  // Anz Ellipsen
  let lc = 55;
  // for-loop > Schleife für mehrere Kreise
  for(let i = 0; i < lc; i++) {

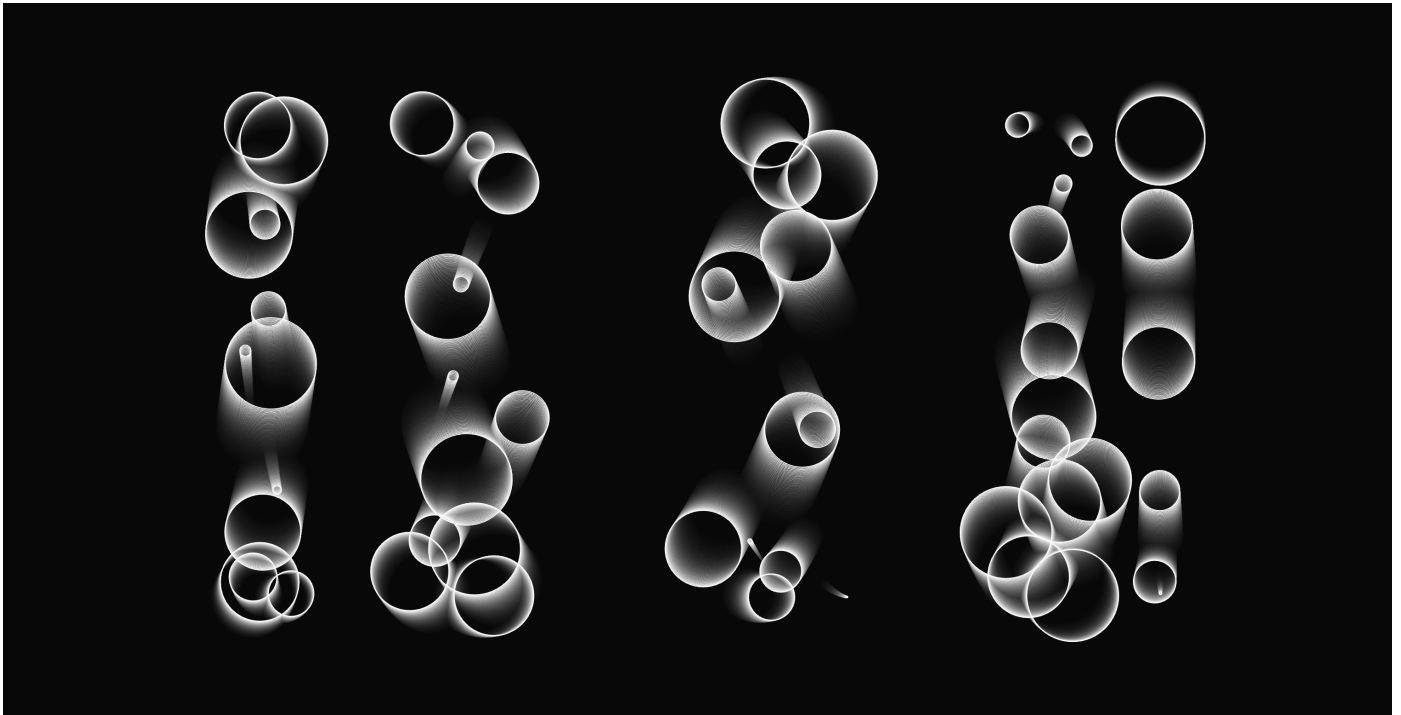
```



```

// X-Pos durch Sinus
// i = versetzte Darstellung
// frameCount = langsame Animation
let x = sin(i*1.4+frameCount*.001)*width/3;
// Y-Pos durch Cosinus
let y = cos(i*6+frameCount*.005)*height/3;
// Grösse der Ellipse
let s = sin(i*.5+frameCount*.0012)*100;
// Zeichnet Ellipsen relativ zur Bildschirmgrösse
ellipse(width/2+x, height/2+y, s)
}
}

```



★ P5.LIVE – Experiment 3

```

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // leicht transparenter HG > Spuren
  background(0, 5)
  stroke(255)
  noFill()

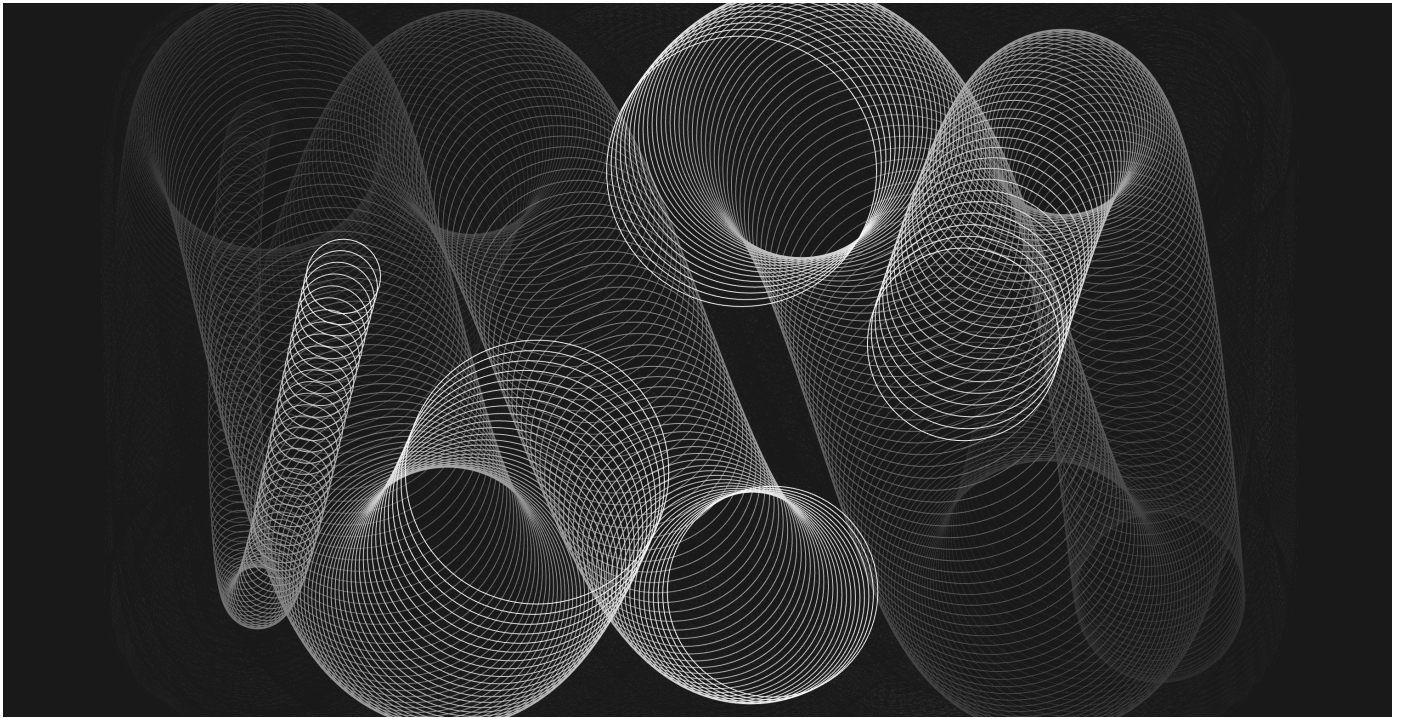
  // Anz Kreise
  let lc = 5;
  // for-loop > mehrere Kreise
  for(let i = 0; i < lc; i++) {
    // X-Pos durch Sinus
    // i = versetzte Darstellung
    // frameCount = langsame Animation
    let x = sin(i*3.4+frameCount*.01)*width/3;
    // Y-Pos durch Cosinus

```

```

let y = cos(i*9+frameCount*.05)*height/3;
// Grösse der Ellipse
let s = sin(i*.5+frameCount*.0012)*300;
// Zeichnet Ellipsen relativ zur Bildschirmgröße
ellipse(width/2+x, height/2+y, s)
}
}

```



★ P5.LIVE – Experiment 4

```

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  // leicht transparenter HG > Spuren
  background(0, 5)
  stroke(255)
  noFill()
  // zeichnet Rechteck von Mitte aus
  rectMode(CENTER)

  // Anz Rechtecke
  let lc = 5;
  // for-Loop > zeichnet mehrere Rechtecke
  for(let i = 0; i < lc; i++) {
    // X-Pos durch Sinus > horizontale Bewegung
    let x = sin(i * 3.4 + frameCount * .001) * width / 2;
    // Y-Pos durch Cosinus > vertikale Bewegung
    let y = cos(i * 9 + frameCount * .005) * height / 2;
    // Grösse der Rechtecke > an Fensterbreite anpassen
    let s = sin(i * .5 + frameCount * .0012) * width;

    // Transformation starten!

```

```
push()  
// Verschiebt Position von Ursprung  
translate(width / 2 + x, height / 2 + y, s)  
// Dreht Rechteck  
rotate(radians(i + 5 + frameCount / 2))  
rect(0,0,s)  
// // Transformation beenden!  
pop()  
}  
}
```

